

Laboratórios de (Informática / Programação) I

2025/2026

Licenciatura em (Engenharia Informática / Ciências da
Computação)



Universidade do Minho

Git

Trabalho em Equipa

O trabalho em equipa pode ser *difícil de gerir*, especialmente quando as equipas são grandes e os sistemas de *partilha de ficheiros* são rudimentares.

Sistemas tradicionalmente usados:

- suportes físicos (pen USB, discos externos);
- email;
- Dropbox, Google Drive, ...

Trabalho em Equipe

Estes meios de partilha são *pouco eficazes* em projetos de programação. São lentos, dificultam trabalho simultâneo e tendem a produzir imensas cópias.

Troca típica de mensagens:

1. projeto.zip
2. projeto2.zip
3. projeto-final.zip
4. projeto-FINAL.zip
5. projeto-final-entrega.zip
6. ...

E a Dropbox/Google Drive?

Estes sistemas reduzem o número de cópias, e permitem uma melhor organização do espaço. No entanto, não lidam bem com *retrocesso de versões*:

“Dropbox keeps snapshots of all changes made to files in your Dropbox within the past 30 days [...]” — Dropbox Help Center

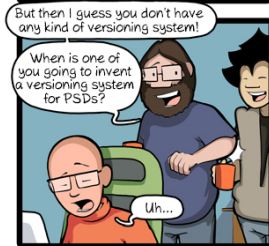
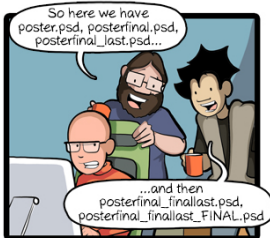
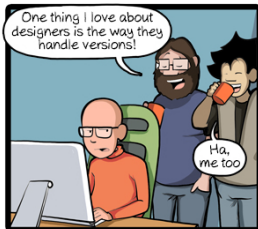
Nem com *resolução de conflitos*:

Name	Date modified
 Sample File (Scott's conflicted copy 2009-10-15)	10/15/2009 4:30 PM
 Sample File	10/15/2009 4:30 PM

Sistemas de Controle de Versões

Estes sistemas são especializados em promover o trabalho colaborativo. Alguns dos seus pontos fortes:

- histórico completo de revisões – o que mudou e quem fez as alterações;
- permitem reverter para versões anteriores;
- permitem resolução de conflitos – automática sempre que possível;
- mensagens descritivas do que mudou em cada versão;
- ramificação, estatísticas, ...



CommitStrip.com

Sistemas de Controle de Versões

Sistemas centralizados:

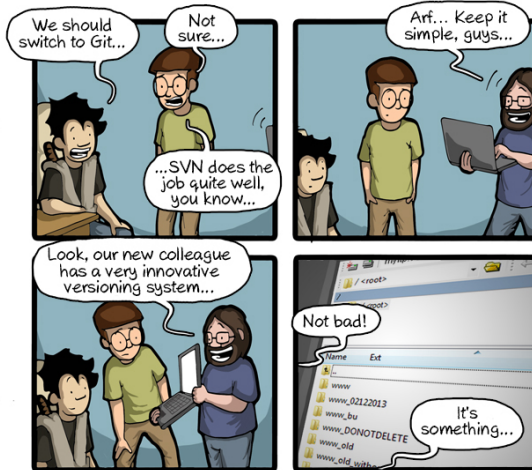
- CVS
- *SVN*

Sistemas distribuídos:

- Git
- Mercurial

Utilizados para manter documentação, ficheiros de configuração e código fonte.

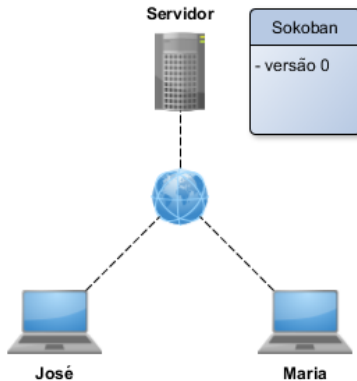
Não é recomendado submeter no repositório ficheiros executáveis.



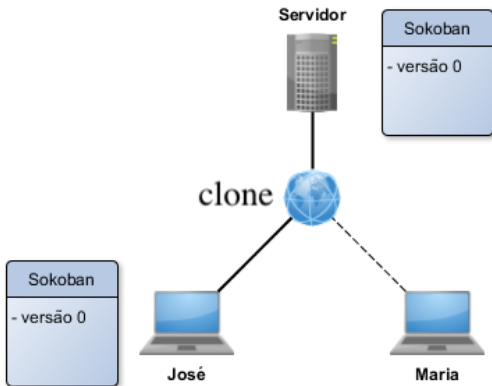
CommitStrip.com



Exemplo



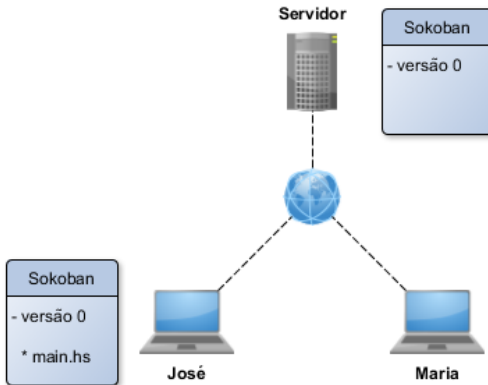
Exemplo



O José:

```
$ git clone git@gitlab.com:/projetos/202511gXXX.git
```

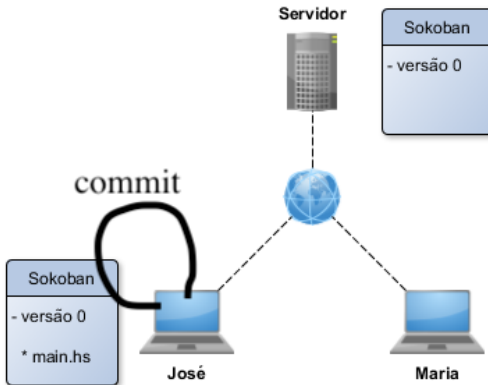
Exemplo



O José:

```
$ code main.hs  
$ git add main.hs
```

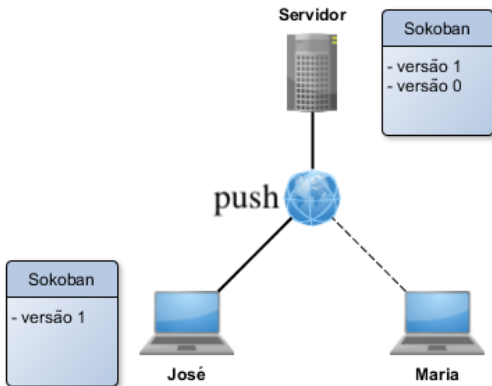
Exemplo



O José:

```
$ git commit -m "ficheiro para resolução do projecto"
```

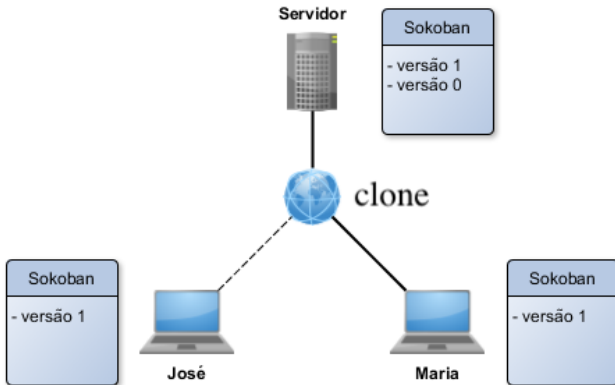
Exemplo



O José:

```
$ git push
```

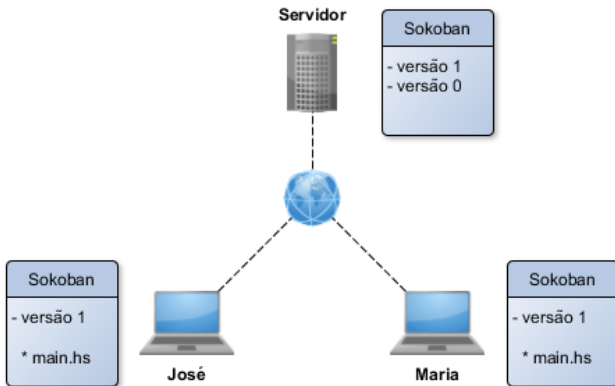
Exemplo



A Maria:

```
$ git clone git@gitlab.com:/projetos/202511gXXX.git
```

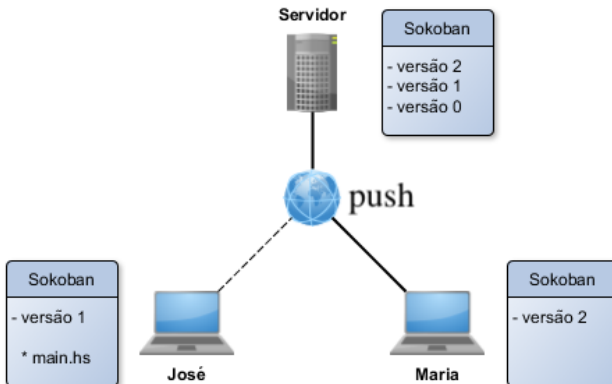
Exemplo



Em paralelo, o José e a Maria:

```
$ code main.hs
```

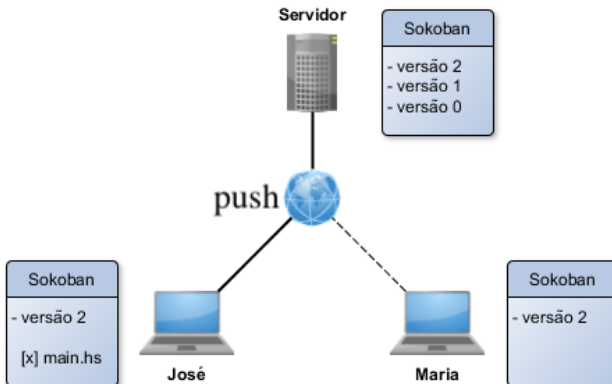

Exemplo



A Maria:

```
$ git add main.hs  
$ git commit -m "proposta de resolucao da tarefa 1"  
$ git push
```

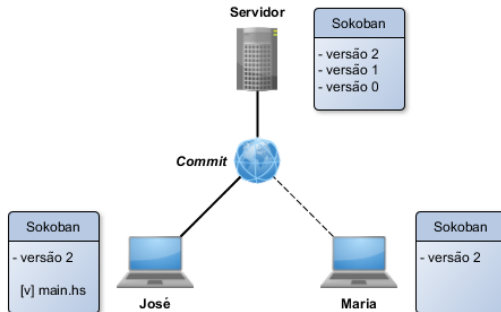
Exemplo



O José:

```
$ git add main.hs  
$ git commit -m "resolvi a tarefa 1"  
$ git push
```

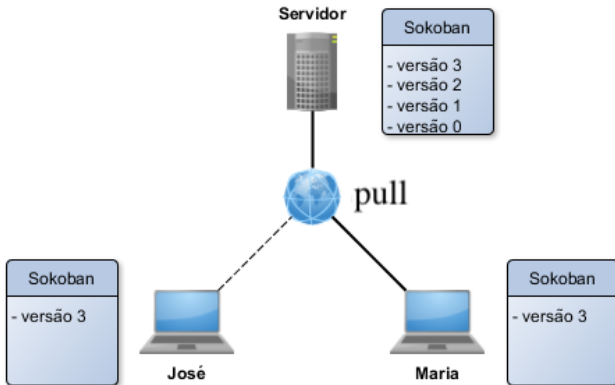
Exemplo



O José:

```
$ git pull
$ code main.hs
$ git add main.hs
$ git commit -m "resolvido conflito"
$ git push
```

Exemplo



A Maria:

```
$ git pull
```

Dicas Git

- Mostrar o log dos commits

```
git log
```

- Verificar conteúdo de uma revisão:

```
git show {revisão}
```

```
$ git show 5227f2263ed7e373e14cf2fe8076f2f695
```

- Verificar diferenças entre revisões:

```
git diff {revisão inicial} {revisão final}
```

```
$ git diff 5227f22 13bd211
```

- Restaurar ficheiro removido/editado acidentalmente:

```
$ git checkout {file}
```